CPU Scheduling

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **23-30**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Draw FIFO, SJF, STCF and RR timelines.

- Calculate turnaround, waiting and first-response time separately.

- Explain MLFQ rules, priority boosts and accounting.

SELECTED PRACTICE

QZ: simultaneous arrivals / original PDF pages 1, 3

Y22-S: different arrival times / original PDF pages 1

Y15-A: MLFQ essay / original PDF pages 6

Each practice section has a source page and question guide. Mixed-topic source pages may appear in more than one pack.

Lecture 3 — Scheduling

อ้างอิง: Lecture 3 หน้า 1–40 | scheduling metrics, FIFO/SJF/STCF/RR, MLFQ และ lottery scheduling

3.1 Scheduling คือ Policy สำหรับแบ่งปันทรัพยากร

> **เนื้อหาจาก Lecture**
>
> Scheduling คือการตัดสินใจว่าจะจัดสรร CPU ให้ process ใด ใน Lecture 2 เราเรียน context switch ซึ่งเป็นกลไกที่ทำให้สลับ process ได้ ส่วน Lecture นี้เน้น policy ที่ใช้เลือก process ให้เหมาะกับเป้าหมายของระบบ

> **เชื่อมกับ Computer Architecture**
>
> Timer Interrupt + Context Switch ทำให้เกิด preemption ได้ แต่ไม่ได้เป็นตัวเลือกว่าจะให้ใครใช้ CPU หน้าที่นั้นเป็นของ Scheduler ซึ่งใช้กลไกดังกล่าวประกอบการตัดสินใจ จำให้ได้ว่า **Mechanism ต่างจาก Policy**

3.1.1 Workload และสมมติฐานของ Job

Lecture เรียกกลุ่ม process ที่ scheduler ต้องจัดการว่า workload หรือ job โดยเริ่มจากแบบจำลองอย่างง่าย: ทุก job ใช้ CPU อย่างเดียว ไม่มี I/O และรู้ run-time ล่วงหน้า จากนั้นจึงค่อยผ่อนสมมติฐานให้ใกล้ระบบจริง

3.2 Scheduling Metrics

เราใช้ metrics วัด performance และ fairness ของ policy บางครั้งการเพิ่ม performance อาจทำให้การแบ่ง CPU ไม่เป็นธรรม จึงต้องพิจารณา trade-off ทั้งสองด้าน

3.2.1 Turnaround Time

$$T_{turnaround} = T_{completion} - T_{arrival} \tag{1}$$

ถ้าทุก job มาถึงพร้อมกันที่เวลา 0: $T_{turnaround} = T_{completion}$.

3.2.2 Response Time

$$T_{response} = T_{first\text{-}run} - T_{arrival} \tag{2}$$

Response Time วัดว่า user/job ต้องรอนานเท่าใดก่อนเริ่มได้รับ CPU ครั้งแรก ไม่ใช่เวลาจนงานเสร็จ

> **จุดที่มักสับสน**
>
> **Response Time ต่างจาก Turnaround Time** policy ที่ดีต่อ metric หนึ่งอาจไม่ดีต่ออีก metric หนึ่ง เช่น SJF/STCF มุ่งลด turnaround ส่วน RR ช่วยให้แต่ละ process เริ่มตอบสนองได้เร็วขึ้น

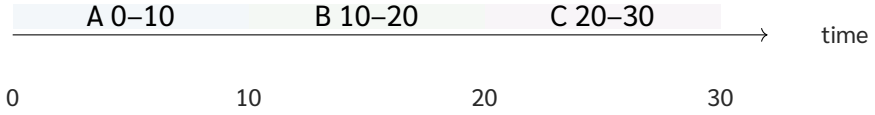
> **รายละเอียดสำคัญจากสไลด์**
>
> **Lecture 3 หน้า 23 — Batch กับ Interactive** งานแบบ batch มักวัดผลจากเวลาที่ทำเสร็จ ส่วนงาน interactive ต้องตอบสนองผู้ใช้ได้เร็ว จึงให้ความสำคัญกับ response time ภาพในสไลด์เชื่อมจาก SJF/STCF/RR ไปสู่ MLFQ ซึ่งพยายามรองรับงานทั้งสองแบบโดยไม่ต้องรู้ job length ล่วงหน้า

3.3 FIFO / FCFS

First In, First Out (FIFO) หรือ First Come, First Served เลือก process ตามลำดับที่มาถึง

ตัวอย่างคำนวณ

ตัวอย่างใน Lecture: A, B และ C มาถึงพร้อมกันที่ $t = 0$ แต่ละ job ใช้ CPU 10 s และเข้าคิวตามลำดับ A, B, C



$T_A = 10$, $T_B = 20$, $T_C = 30$ s.

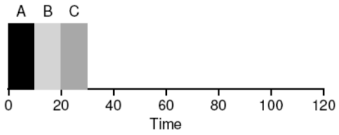
$$\overline{T}_{turnaround} = \frac{10 + 20 + 30}{3} = 20 \text{ s}$$

ภาพประกอบ 3.1 Lecture 3 หน้า 7 — FIFO / FCFS Timeline

First In, First Out (FIFO)

(First Come, First Served)

- Imagine three jobs arrive in the system, A, B, and C, at the same time ($T_{arrival}$ = 0)
- FIFO has to put some job first, let's assume A -> B -> C
- Assume also that each job runs for 10 seconds
- What will the average turnaround time be for these jobs?



(10+20+30)/3 = 20 seconds

จุดสังเกต

- Job ถูกวางตาม arrival order; ไม่มีการเลือกจาก job length
- ใช้ timeline อ่าน completion time ของแต่ละ job แล้วคำนวณ turnaround
- ความง่ายของ FIFO ไม่ได้แปลว่า metric ดีทุก workload

ที่มา: Lecture 3 หน้า 7.

3.3.1 ข้อจำกัดของ FIFO: Convoy Effect

สมมติว่า A ใช้เวลา 100 s ส่วน B และ C ใช้เวลา 10 s และมาถึงพร้อมกัน หาก FIFO เรียงเป็น A, B, C งานสั้นทั้งสองต้องรออยู่หลังงานยาว:

$$\overline{T}_{turnaround} = \frac{100 + 110 + 120}{3} = 110 \text{ s}$$

ปัญหาคือ resource consumer สั้น ๆ ถูก queue อยู่หลัง long job.

3.4 Shortest Job First (SJF)

Shortest Job First (SJF) เลือก job ที่ใช้เวลาสั้นที่สุดก่อน ภายใต้สมมติฐานว่าทุก job มาถึงพร้อมกัน SJF ให้ average turnaround ต่ำที่สุดตาม Lecture

สำหรับ A=100, B=10, C=10 ถ้ารันตามลำดับ B, C, A:

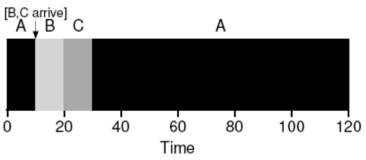
$$\overline{T}_{turnaround} = \frac{10 + 20 + 120}{3} = 50 \text{ s}$$

หมายเหตุในสไลด์หน้า 9 ชี้ว่าในตัวอย่างนี้ SJF ให้ average turnaround ต่ำกว่า FIFO

ภาพประกอบ 3.2 Lecture 3 หน้า 11 — STCF with Different Arrival Times

STCF's example

- A arrives at t = 0 and needs 100 secs, B and C arrive at t = 10, each need 10 secs



- Ave turnaround time (120+10+20)/3 = 50
- In batch scheduling, this make sense
- However, in time-shared machines where users sit at a terminal and demand interactive performance, a new metric was introduced: response time

จุดสังเกต

- A เริ่มทำงานก่อน เมื่อ B และ C มาถึง scheduler จะเปรียบเทียบ **remaining time**
- หากงานใหม่มี remaining time สั้นกว่า A, STCF สามารถ preempt A ได้
- จุดสอบคือแยก total job length ของ SJF กับ remaining time ณ ตอนตัดสินใจของ STCF

ที่มา: Lecture 3 หน้า 11.

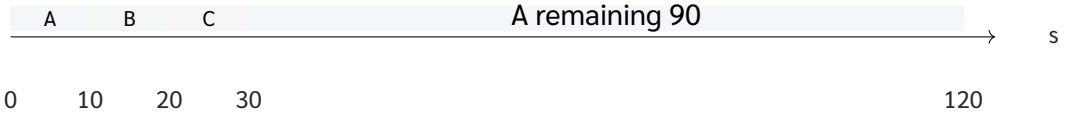
3.5 เมื่อ Arrival Time ต่างกัน: STCF / Preemptive SJF

ในตัวอย่างถัดมา Lecture เปลี่ยนเงื่อนไขให้ A มาถึงที่ $t = 0$ และใช้เวลา 100 s ส่วน B กับ C มาถึงที่ $t = 10$ และใช้เวลา 10 s เท่ากัน หาก scheduler ไม่ preempt A ค่า average turnaround จะเป็น 103.33 s ตามสไลด์

Shortest Time-to-Completion First (STCF) หรือ Preemptive SJF ตรวจ remaining time เมื่อมี job ใหม่เข้ามา และรัน job ที่เหลือเวลาน้อยที่สุด

ตัวอย่างคำนวณ

Timeline STCF:



เวลาที่แต่ละ job เสร็จคือ A=120, B=20 และ C=30 ส่วนเวลาที่มาถึงคือ A=0 และ B=C=10

$$T_A = 120, \quad T_B = 10, \quad T_C = 20$$

$$\overline{T}_{turnaround} = \frac{120 + 10 + 20}{3} = 50 \text{ s}$$

ดังนั้น response time ของ A และ B เท่ากับ 0 เพราะเริ่มทำงานทันทีเมื่อมาถึง ส่วน C เริ่มที่ $t = 20$ หลังมาถึงที่ $t = 10$ จึงมี response time เท่ากับ 10 s

$$\overline{T}_{response} = \frac{0 + 0 + 10}{3} = 3.33 \text{ s}$$

> **เชื่อมกับ Computer Architecture**
>
> STCF ต้องอาศัย preemption Lecture ระบุว่า timer interrupt และ context switch ช่วยให้ scheduler หยุด A แล้วเปลี่ยนไปรัน job ใหม่ได้ นี่คือตัวอย่างที่ hardware mechanism รองรับการทำงานของ OS policy

3.6 Round Robin (RR)

Round Robin (RR) ไม่ปล่อยให้ job หนึ่งรันจนเสร็จ แต่ให้แต่ละ job ใช้ CPU ตาม **time slice / quantum** แล้วสลับไป job ถัดไปใน run queue ทำซ้ำจนเสร็จ วิธีนี้ช่วยแบ่งเวลาใช้ CPU ให้ใกล้เคียงกันในช่วงสั้น
Lecture ระบุว่า time slice ต้องเป็นจำนวนเท่าของ timer-interrupt period ตัวอย่างเช่น หาก timer interrupt เกิดทุก 10 ms ก็เลือก quantum ได้เป็น 10, 20 ms และค่าที่เป็นเท่าต่อไป

> **ตัวอย่างคำนวณ**
>
> ตัวอย่างเปรียบเทียบใน Lecture ให้ A, B และ C มาถึงพร้อมกัน โดยแต่ละ job ต้องการ CPU 5 s
> สำหรับ SJF เวลาเริ่มทำงานครั้งแรกคือ 0, 5 และ 10 s จึงได้ average response $= (0 + 5 + 10)/3 = 5$ s
> สำหรับ RR (ตามผลในสไลด์ ใช้ time slice ที่ทำให้ first-run offsets เป็น 0, 1 และ 2):
>
> $$\overline{T}_{response} = \frac{0 + 1 + 2}{3} = 1 \text{ s}$$
>
> RR จึงตอบสนองได้เร็วกว่า แม้ turnaround โดยทั่วไปจะไม่ใช่จุดแข็งของวิธีนี้

ภาพประกอบ 3.3 Lecture 3 หน้า 14 — Round Robin

Round Robin (RR)



- Instead of running jobs to completion, RR
  - runs a job for a time slice
  - then switches to the next job in the run queue
  - repeatedly does so until all jobs are finished
- RR is considered ‘fair’, as it evenly divides the CPU among active processes on a small time scale
- Note that the length of a time slice must be a multiple of the timer-interrupt period
  - If the timer interrupts every 10 milliseconds, the time slice could be 10, 20, or any other multiple of 10 ms

จุดสังเกต

- RR แบ่ง CPU เป็น time slice แล้วหมุนไปตาม run queue
- quantum สั้นช่วยลด first response time แต่ทำให้เกิด context switch บ่อยขึ้น
- quantum ที่ยาวมากทำให้ RR มีพฤติกรรมใกล้ FIFO

ที่มา: Lecture 3 หน้า 14.

3.6.1 Quantum trade-off และ Amortization

- quantum สั้นช่วยให้ตอบสนองเร็ว แต่ context-switch overhead เกิดบ่อย
- quantum ยาวช่วยเฉลี่ย switch cost ต่อเวลาทำงานได้ดีขึ้น แต่ทำให้ระบบตอบสนองช้าลง

ตัวอย่างในสไลด์กำหนด quantum 10 ms และ context switch 1 ms จึงมี overhead ราว 10% หากเพิ่ม quantum เป็น 100 ms overhead จะลดลงเหลือประมาณ 1% ตามแบบจำลองใน Lecture

> **จุดที่มักสับสน**
>
> อย่าสรุปว่า quantum ยิ่งสั้นยิ่งดีโดยไม่กล่าวถึง overhead การเลือก quantum ต้องสมดุลระหว่าง responsiveness กับต้นทุนของ context switch

3.7 Trade-off ระหว่าง Turnaround กับ Response

Lecture แยกแนวโน้มของแต่ละ policy ไว้ชัดเจน:

- SJF/STCF เลือกงานที่เหลือเวลาสั้น จึงมุ่งลด turnaround แต่อาจตอบสนองต่อ job อื่นช้า
- RR สลับให้แต่ละ job ได้ CPU จึงมุ่งลด response time แต่อาจไม่ได้ลด turnaround

นี่คือตัวอย่างของ system-design trade-off เพราะไม่มี policy เดียวที่ให้ผลดีที่สุดกับทุก metric และทุก workload

3.8 นำ I/O มาพิจารณาในการจัดตาราง

เมื่อ job เริ่ม I/O ก็ไม่ต้องใช้ CPU และจะเปลี่ยนเป็น Blocked ระหว่างนั้น scheduler สามารถให้ process อื่นทำงานได้ เมื่อ I/O เสร็จ hardware interrupt จะแจ้ง OS ให้ย้าย process กลับเป็น Ready

> **เชื่อมกับ Computer Architecture**
>
> Computer Architecture อธิบายว่า hardware ส่ง IRQ เพื่อเรียก ISR หรือ handler ก่อน return ส่วน OS เชื่อมเหตุการณ์นี้กับ process state: เมื่อเริ่ม I/O process จะเป็น **Blocked** และเมื่อ device completion interrupt เกิดขึ้นจะกลับเป็น **Ready**

ตัวอย่างใน Lecture ให้ A และ B ต้องการ CPU คนละ 50 ms โดย A ใช้ CPU ครั้งละ 10 ms สลับกับ I/O ครั้งละ 10 ms ส่วน B ใช้ CPU ต่อเนื่อง หากรอให้ A ทำงานทั้งหมดจบก่อนแล้วค่อยให้ B เริ่ม ระบบจะพลาดโอกาสทำ CPU และ I/O ซ้อนกัน Scheduler จึงสลับไปรัน B ระหว่างที่ A รอ I/O ได้

3.9 เมื่อไม่รู้ Job Length ล่วงหน้า: Multi-Level Feedback Queue (MLFQ)

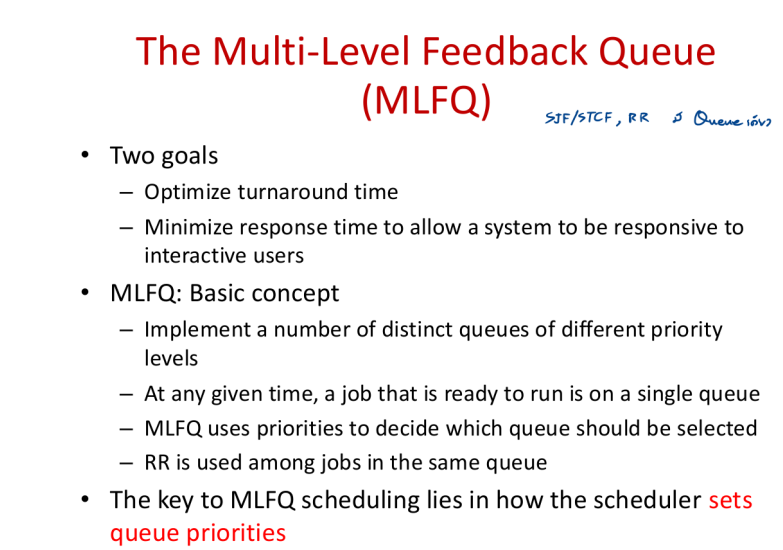
ในระบบจริง OS ไม่รู้ว่า job จะใช้ CPU นานเท่าไร MLFQ จึงพยายามตอบโจทย์สองด้านพร้อมกัน:

- ลด turnaround โดยให้งานสั้นได้ priority สูง
- ลด response time สำหรับผู้ใช้ interactive

MLFQ มีหลาย queue ซึ่งกำหนด priority ต่างกัน process ที่อยู่ในสถานะ Ready จะอยู่ใน queue เดียวในแต่ละขณะ Scheduler เลือกจาก queue ที่มี priority สูงก่อน และใช้ RR เมื่อมีหลาย process อยู่ใน queue ระดับเดียวกัน

> **อ่านภาพและหมายเหตุ**
>
> สไลด์หน้า 22 มีหมายเหตุเขียนมือว่า “สอบแน่นอน” ไว้ใกล้คำถามเรื่อง OS ไม่รู้ job length และคำตอบ MLFQ จึงควรฝึกอธิบายแนวคิดและไล่การทำงานของ MLFQ ให้คล่อง

ภาพประกอบ 3.4 Lecture 3 หน้า 24 — Multi-Level Feedback Queue



จุดสังเกต

- MLFQ มีหลาย priority queues โดยเลือก queue ที่มี priority สูงก่อน
- feedback มาจากพฤติกรรมการใช้ CPU ไม่ใช่ job length ที่รู้ล่วงหน้า
- ภายใน queue ที่มี priority เท่ากัน ใช้ RR ตามกฎใน Lecture

ที่มา: Lecture 3 หน้า 24.

3.9.1 ใช้พฤติกรรมที่สังเกตได้กำหนด Priority

MLFQ ปรับ priority ตามพฤติกรรมที่สังเกตได้:

- job ที่รอ keyboard หรือ I/O บ่อยมักมีพฤติกรรม interactive จึงได้รับ priority สูง
- job ที่ใช้ CPU ต่อเนื่องเป็นเวลานานจะถูกลด priority

3.9.2 Final MLFQ Rules ตาม Lecture

1. ถ้า Priority(A) > Priority(B) จะให้ A ใช้ CPU ก่อน B
2. ถ้า Priority(A) = Priority(B) จะแบ่ง CPU ด้วย RR
3. เมื่อมี job ใหม่เข้าระบบ ให้วางไว้ใน queue ที่มี priority สูงสุด
4. เมื่อ job ใช้ **time allotment** ของ queue นั้นครบแล้ว ให้ลด priority ลงหนึ่งระดับ ไม่ว่า job จะคืน CPU ก่อนหมด quantum กี่ครั้งก็ตาม
5. เมื่อครบช่วงเวลา S ให้ย้ายทุก job กลับไปยัง queue ที่มี priority สูงสุด (priority boost)

ภาพประกอบ 3.5 Lecture 3 หน้า 35 — Final MLFQ Rules

The Final Rules

- ***Rule 1:*** If Priority(A) > Priority(B), A runs (B doesn't)
- ***Rule 2:*** If Priority(A) = Priority(B), A & B run in RR
- ***Rule 3:*** When a job enters the system, it is placed at the highest priority (the topmost queue)
- ***Rule 4:*** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e. it moves down one queue)
- ***Rule 5:*** After some time period S, move all the jobs in the system to the topmost queue

จุดสังเกต

- Rule 1–2 = เลือก priority queue และใช้ RR เมื่อ priority เท่ากัน
- Rule 3 = new job เริ่มบนสุด
- Rule 4 = ใช้ time allotment สะสม ปิดช่องgaming
- Rule 5 = periodic priority boost แก้ starvation / เปิดทางให้ behavior ที่เปลี่ยนถูกประเมินใหม่

ที่มา: Lecture 3 หน้า 35 ใช้ทบทวนกฎ MLFQ ฉบับสุดท้าย

> **อธิบายให้เห็นภาพ**
>
> กฎข้อ 4 ฉบับสุดท้ายแก้ช่องโหว่ของกฎรุ่นก่อน หากนับเฉพาะการใช้ quantum จนหมด process อาจคืน CPU ก่อนหมด quantum ซ้ำ ๆ เพื่อหลีกเลี่ยงการลด priority กฎใหม่จึงสะสมเวลา CPU ที่ใช้ในแต่ละระดับจนครบ time allotment

3.9.3 ปัญหาที่ MLFQ ต้องป้องกัน

Starvation: หากมีงาน interactive จำนวนมาก งาน batch ที่ใช้เวลานานอาจไม่ได้ใช้ CPU กฎ priority boost จึงช่วยให้ทุก job ได้เริ่มต้นใหม่ในระดับสูงสุด

Gaming: process อาจพยายาม yield ก่อน quantum หมดเพื่อหลีกเลี่ยงการลด priority กฎ time allotment ป้องกันวิธีนี้โดยนับเวลา CPU สะสมในแต่ละระดับ

พฤติกรรมที่เปลี่ยนไป: งาน CPU-bound อาจกลายเป็นงาน interactive ได้ การ boost เป็นระยะช่วยให้ scheduler ประเมินพฤติกรรมของงานใหม่

จำนวน queue, quantum ของแต่ละ queue และช่วงเวลา S ต้องปรับให้เหมาะกับ workload Lecture ไม่ได้กำหนดค่าชุดเดียวที่ใช้ได้กับทุกระบบ

จุดเน้นก่อนสอบ

เมื่อทบทวน MLFQ ควรเขียนกฎทั้ง 5 ข้อและไล่ให้ได้ว่า job เลื่อน queue เพราะอะไร ควรอธิบายวิธีแก้ starvation และ gaming ได้ด้วย รวมทั้งชี้ให้เห็นว่า MLFQ ใช้พฤติกรรมที่สังเกตได้มาประมาณ job length ที่ระบบไม่รู้ล่วงหน้า

3.10 Proportional-Share และ Lottery Scheduling

ช่วงท้ายของ Lecture เปลี่ยนเป้าหมายมาเป็นการแบ่ง CPU ตามสัดส่วน Proportional-Share Scheduler จึงพยายามให้แต่ละ job ได้ส่วนแบ่ง CPU ตามจำนวน ticket ที่ถือ

3.10.1 Tickets

Ticket ใช้แทนส่วนแบ่ง CPU หาก A มี 75 tickets และ B มี 25 tickets จากทั้งหมด 100 ใบ เมื่อมองในระยะยาว A ควรได้ CPU ราว 75% และ B ราว 25%

3.10.2 Holding a Lottery

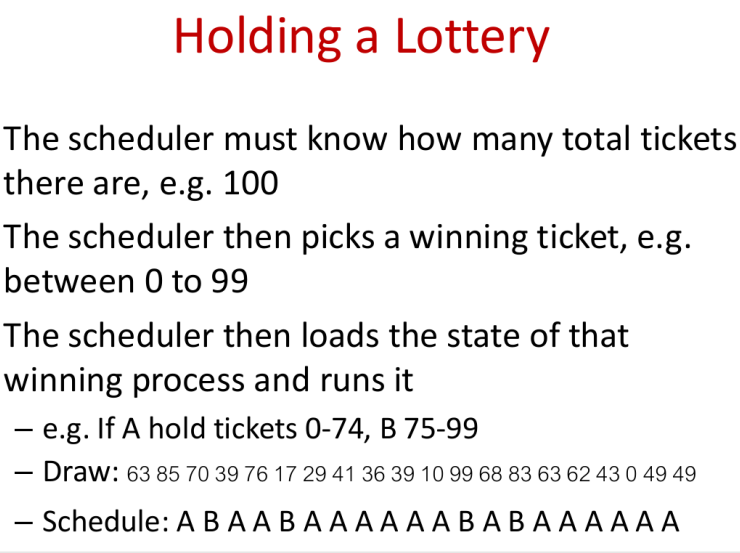
ในแต่ละรอบการจัดตารางหรือ time slice:

1. scheduler รู้จำนวน ticket รวม เช่น 100
2. สุ่มหมายเลข ticket ที่ชนะระหว่าง 0–99
3. ตรวจว่า ticket นั้นเป็นของ process ใด
4. คืน state ของ process ที่ชนะ แล้วให้ใช้ CPU

ตัวอย่างใน Lecture กำหนดให้ A ถือ ticket หมายเลข 0–74 และ B ถือ 75–99 หากสุ่มได้ลำดับ 63, 85, 70, 39, 76, ... ผู้ชนะในแต่ละรอบจะแปรผัน แต่สัดส่วนการได้ CPU จะเข้าใกล้จำนวน ticket เมื่อดูในระยะยาว

ตาม Lecture วิธีสุ่มช่วยหลีกเลี่ยงกรณีขอบบางแบบ ใช้ state ที่ต้องติดตามไม่มาก และทำงานได้รวดเร็ว อย่างไรก็ตาม ผลลัพธ์ขึ้นอยู่กับการจัดสรร ticket มาก Lecture ยกตัวอย่างให้ user ได้รับ ticket แล้วแบ่งให้ job ของตน หรือใช้วิธีจัดสรรที่ซับซ้อนขึ้น

ภาพประกอบ 3.6 Lecture 3 หน้า 39 — Holding a Lottery



จุดสังเกต

- รวม ticket ทั้งหมดแล้วสุ่มหมายเลขที่ชนะ
- process ที่ถือ ticket หมายเลขนั้นได้ CPU ในรอบนี้
- สัดส่วนการใช้ CPU จะเข้าใกล้สัดส่วน ticket ในระยะยาว แต่ไม่จำเป็นต้องเป็น 75/25 ในทุกช่วงสั้น

ที่มา: Lecture 3 หน้า 39.

3.11 Scheduling Comparison

นโยบาย	แย่ง CPU ได้หรือไม่	หลักการ	จุดเด่นและข้อจำกัด
FIFO	ไม่แย่ง	เรียง ตาม เวลามาถึง	เข้าใจง่าย แต่งานสั้นอาจต้องรอหลังงานยาว
SJF	ไม่แย่ง	งานที่ใช้เวลาสั้นที่สุด	ลด average turnaround ได้เมื่อทุกงานมาพร้อมกัน แต่ต้องรู้ความยาวงาน
STCF	แย่งได้	เวลาที่เหลือน้อยที่สุด	ลด turnaround เมื่อเวลา arrival ต่างกัน แต่อาจตอบสนองช้า
RR	แย่งได้	แบ่ง เวลา เป็น quantum	ตอบสนองเร็ว แต่มี context-switch overhead
MLFQ	แย่งได้	feedback และ priority queue	ประมาณพฤติกรรมของงานโดยไม่รู้ job length แต่ต้องป้องกัน starvation และ gaming
Lottery	แย่ง ได้ เมื่อแบ่งเวลา	สุ่มตาม ticket	ให้สัดส่วน CPU ในระยะยาวตามจำนวน ticket

3.12 Chapter Summary

สรุปท้ายบท

- Scheduling เป็น policy ส่วน timer interrupt และ context switch เป็น mechanism metrics หลัก คือ turnaround และ response
- FIFO เรียบง่าย SJF/STCF มุ่งลด turnaround ส่วน RR มุ่งให้ตอบสนองเร็ว โดยต้องคำนึงถึง quantum และ context-switch overhead
- เมื่อเริ่ม I/O process จะเป็น Blocked และกลับ Ready เมื่อ completion interrupt มาถึง ทำให้ CPU กับ I/O ทำงานซ้อนกันได้
- MLFQ ใช้ feedback และ priority เพื่อประมาณพฤติกรรม interactive กับ CPU-bound พร้อมกฎป้องกัน starvation และ gaming
- Lottery scheduling ใช้ ticket และการสุ่มเพื่อจัดสรร CPU ตามสัดส่วน

Formula / Calculation Recap

$$T_{turnaround} = T_{completion} - T_{arrival}, \qquad T_{response} = T_{first\text{-}run} - T_{arrival}$$

ค่าเฉลี่ยของ metric = ผลรวมของทุก job หารด้วยจำนวน job ควรวาด timeline ก่อนคำนวณเสมอ

ComArch Connections

Timer interrupt ทำให้เกิด preemption ได้ การ save/restore CPU state เป็นต้นทุนของ context switch และ I/O completion interrupt ทำให้ process เปลี่ยนจาก Blocked เป็น Ready

Common Confusions

- STCF ต่างจาก SJF และ response time ต่างจาก turnaround time
- RR quantum ต่างจาก timer period โดย quantum ต้องเป็นจำนวนเท่าของ timer period ตาม Lecture
- MLFQ priority boost ย้ายทุก job ไป queue บนสุด ส่วนกฎข้อ 4 นับ time allotment สะสมเพื่อป้องกัน gaming

Quick Revision Checklist

- ☐ วาด timeline ของ FIFO/SJF/STCF/RR และคำนวณค่าเฉลี่ย
- ☐ อธิบาย quantum กับ context-switch overhead
- ☐ ไล่ I/O จาก Blocked ผ่าน interrupt กลับ Ready
- ☐ ทวนกฎ MLFQ และวิธีแก้ starvation/gaming
- ☐ อธิบายสัดส่วน ticket 75/25

Practice: QZ: simultaneous arrivals

ORIGINAL SOURCE

Quiz_BeforeMidterm.pdf

Original PDF page(s): **1, 3**

Page 1 uses quantum 1; page 3 uses quantum 10. Check the metric requested.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

66070501034 เฟิมกา

1. Consider the following set of processes (jobs), where the run time of the process is given in milliseconds (ms):

Process	Run Time
P_1	10
P_2	1
P_3	2
P_4	2
P_5	5

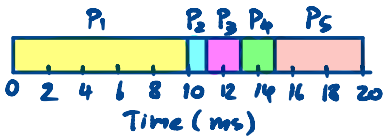
The processes are assumed to arrive in the order P_1, P_2, P_3, P_4, P_5, all at time 0.

a) Draw two diagrams illustrating the execution of these processes using FIFO, SJF~~, and RR scheduling with time slice = 1 ms.~~

b) What is the Turnaround time of each process for each of the schedulers in part (a)?

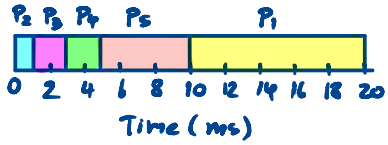
$T_{turnaround} = T_{completion} - T_{arrival}$ (0)

FIFO



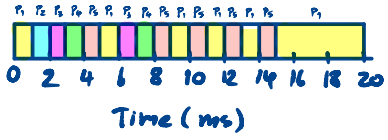
$T = \frac{10+11+13+15+20}{5} = 13.8$ ms #

SJF



$T = \frac{20+10+5+3+1}{5} = 7.8$ ms #

RR



$T = \frac{20+2+7+8+15}{5} = 10.4$ ms #

Question 2: (10 points)

6. **(3 points)** Shown below is the workload for 5 jobs arriving at time zero in the order given below

Job	Execution time
1	10
2	29
3	3
4	7
5	12

Calculate the average waiting time if three different algorithms are used: FCFS, SJF (non-preemptive), and Round Robin with time slice = 10.

a) FCFS

ANS:

Job	waiting time
1	0
2	10
3	39
4	42
5	49



The average waiting time = $\frac{10+39+42+49}{5} = 28$

b) SJF

ANS:

Job	waiting time
1	10
2	32
3	0
4	3
5	20



The average waiting time = $\frac{10+32+3+20}{5} = 13$

c) RR

ANS:

Job	waiting time
1	0
2	61-29: 32
3	23-3: 20
4	30-7: 23
5	52-12: 40



The average waiting time = $\frac{32+20+23+40}{5} = 23$

Practice: Y22-S: different arrival times

ORIGINAL SOURCE

OS-Midterm-2022-p2-1.pdf

Original PDF page(s): **1**

FCFS, preemptive SJF/STCF and RR. See CHECKED_DRILLS for recomputed values and tie/queue assumptions.

ANSWER ANNOTATIONS

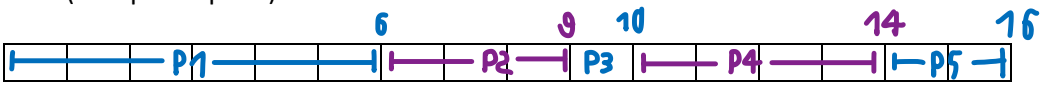
Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

Midterm 1/2022 – Part 2.1

1. Given the information below, draw grant charts for the algorithms. Also calculate average waiting time and average turnaround time for each algorithm. (12 pts)

Process	Arrival Time	Burst Time
P1	0.0	6
P2	2.0	3
P3	3.0	1
P4	4.0	4
P5	6.0	2

a. FCFS (non-preemptive)



Average waiting time 4.8 Average Turnaround time 8

b. SFJ (preemptive)



Average waiting time 3 Average Turnaround time 6.2

c. RR (quantum = 1)



Average waiting time 5.6 Average Turnaround time 8.8

d. Which algorithm has
   - Smallest average waiting time SFJ
   - Largest average waiting time RR
   - Smallest average turnaround time SFJ
   - Largest average turnaround time RR

Practice: Y15-A: MLFQ essay

ORIGINAL SOURCE

Y15-A / 2015 Operating Systems exam, annotated copy A (9 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **6**

Explain the bank example by mapping decisions to the five final MLFQ rules.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

ชื่อ .....รหัสประจำตัว.....ภาควิชา/ชั้นปี.....

II. (15 points) Explain how a commercial bank handles customers during rush hours with the concept of the Multi-level Feedback Queue (MLFQ), in your opinion. Describe the algorithms and policies used and provide suggestions on how to improve the process with the goal of optimizing the average turnaround time of customers.